

CSS Grid: Guía Teórica Completa

1. Qué es CSS Grid

CSS Grid es un sistema de layout bidimensional que permite organizar elementos en filas y columnas con un control total. Está diseñado para construir estructuras complejas con precisión milimétrica y menos caos que cualquier alternativa anterior.

2. Conceptos Fundamentales

Grid container

Un elemento se convierte en grid al aplicar:

```
display: grid;  
display: inline-grid;
```

Grid items

Son los hijos directos del contenedor.

Filas y columnas

Grid trabaja simultáneamente en ambas dimensiones.

3. Definir Filas y Columnas

grid-template-columns

```
grid-template-columns: 200px 1fr 2fr;  
grid-template-columns: repeat(3, 1fr);  
grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
```

grid-template-rows

```
grid-template-rows: 100px auto 50px;
```

grid-template (shorthand)

```
grid-template: repeat(2, 200px) / repeat(3, 1fr);
```

auto-fill vs auto-fit

- **auto-fill:** llena el espacio incluso si sobran columnas vacías.
 - **auto-fit:** ajusta el contenido sin generar columnas fantasma.
-

4. Espaciado y Gaps

```
gap: 1rem;  
row-gap: 2rem;  
column-gap: 1rem;
```

5. Ubicación de Items

grid-column y grid-row

```
grid-column: 1 / 3;  
grid-row: 2 / span 2;
```

shorthand: place-self

```
place-self: center;
```

Alineación del grid

```
justify-items: start | end | center | stretch;  
align-items: start | end | center | stretch;  
  
justify-content: start | end | center | space-between | space-around | space-evenly;  
align-content: start | end | center | space-between | space-around | stretch;
```

6. Grid Implícito

Cuando los items exceden la cantidad definida de filas o columnas:

grid-auto-rows / grid-auto-columns

```
grid-auto-rows: 150px;  
grid-auto-columns: minmax(100px, auto);
```

grid-auto-flow

```
grid-auto-flow: row;  
grid-auto-flow: column;  
grid-auto-flow: dense; /* rellena huecos automáticamente */
```

7. Funciones Importantes

minmax()

Define un rango flexible:

```
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

repeat()

Repetición compacta:

```
repeat(4, 1fr);
```

fr (fractional unit)

Unidad clave para layouts fluidos.

8. Áreas del Grid

grid-template-areas

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr 200px;  
  grid-template-areas:  
    "header header header"  
    "sidebar content ads"  
    "footer footer footer";  
}  
  
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.content { grid-area: content; }  
.ads { grid-area: ads; }  
.footer { grid-area: footer; }
```

Permite layouts semánticos y mantenibles.

9. Patrones Comunes

Layout de tres columnas fluido

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 1rem;  
}
```

Gallery responsive automática

```
.gallery {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));  
  gap: 1rem;  
}
```

Centered card

```
.wrapper {  
  display: grid;  
  place-items: center;  
}
```

10. Buenas Prácticas

- Usar `minmax()` para responsive sin media queries.
- Combinar `auto-fit` con `repeat` para grillas fluidas.
- No abusar de `grid-template-areas` en layouts gigantes.
- Evitar especificar filas y columnas rígidas cuando el contenido varía.
- Aprovechar `dense` si quieres maximizar espacio.

11. Grid vs Flexbox

- **Grid:** bidimensional, ideal para estructuras globales.
- **Flexbox:** unidimensional, mejor para componentes.

No compiten, se complementan.

12. Checklist Rápido

- [] ¿Definiste el número de columnas?
- [] ¿Tus filas requieren altura fija?
- [] ¿Agregaste `gap`?
- [] ¿Necesitas `auto-fit` o `auto-fill`?
- [] ¿Tus items están bien posicionados con `grid-column` o `grid-row`?

13. Para Practicar

- Dashboard responsive
- Landing page con hero y cards
- Portfolio con grilla autoajutable
- Mock de panel administrativo

14. Objetivo

Este archivo consolida toda la teoría clave de CSS Grid para tener un toolkit sólido y escalable al construir layouts profesionales.